



**KOM120C : PEMROGRAMAN**

## **K08 | Pekan Ke-09** **Pemrograman Fungsional**

---

Tim Pengajar Pemrograman IPB University

# Learning Outcome

- Mahasiswa memahami paradigma pemrograman fungsional.
- Mahasiswa mengenal pemrograman fungsional menggunakan Scala.
- Mahasiswa memahami konsep fungsi dalam pemrograman fungsional.
- Mahasiswa mengenal struktur pemrograman Scala.





# Catatan UTS

Mahasiswa kurang memahami konsep dan implementasi OOP secara utuh !

Perlu meluangkan lebih banyak waktu untuk **membaca** !

Perlu memperluas sumber-sumber pembelajaran, tidak hanya dari bahan kuliah.

## KULIAH

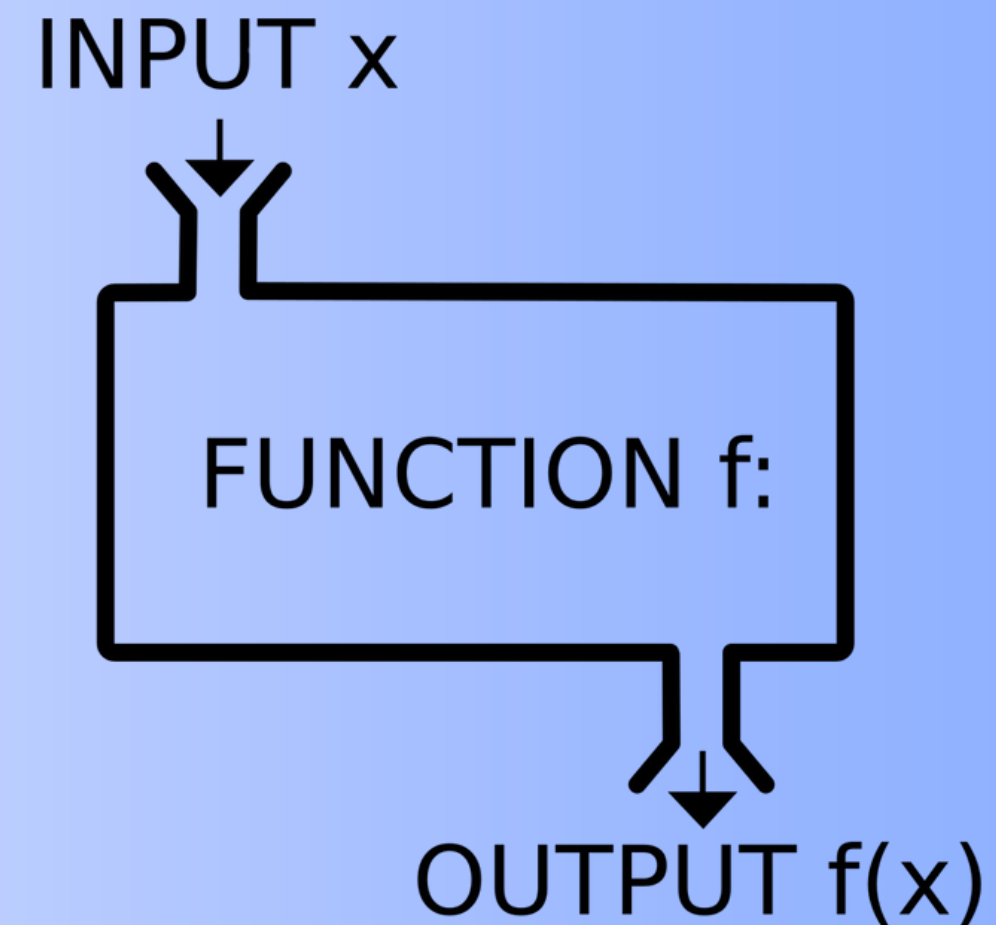
| Statistik                      | K01  | K02  | K03  | K04  | K05  | K06  | ALL  |
|--------------------------------|------|------|------|------|------|------|------|
| Rata-rata                      | 29.6 | 33.7 | 37.0 | 36.5 | 23.7 | 26.2 | 31.1 |
| Nilai Minimum                  | 8.0  | 9.6  | 11.2 | 13.6 | 0.0  | 8.8  | 0.0  |
| Nilai Maksimum                 | 78.8 | 74.0 | 69.6 | 66.0 | 48.4 | 56.8 | 78.8 |
| Std. Deviasi                   | 15.8 | 17.1 | 13.2 | 12.3 | 11.0 | 13.9 | 15.1 |
| Persen di Bawah Rata-rata Umum | 60.0 | 50.0 | 45.8 | 62.5 | 78.8 | 65.4 | 55.9 |

## PRAKTIKUM

| KELAS | N   | Rataan | StdDev | Min | Max   | % Nilai 0 |
|-------|-----|--------|--------|-----|-------|-----------|
| K01   | 40  | 39.2   | 31.9   | 0.0 | 100.0 | 5.9       |
| K02   | 40  | 38.4   | 38.9   | 0.0 | 100.0 | 7.8       |
| K03   | 24  | 36.9   | 37.2   | 0.0 | 100.0 | 4.9       |
| K04   | 40  | 52.2   | 33.5   | 0.0 | 100.0 | 2.9       |
| K05   | 33  | 22.2   | 28.8   | 0.0 | 100.0 | 8.8       |
| K06   | 27  | 11.8   | 22.9   | 0.0 | 76.5  | 9.8       |
| TOTAL | 204 | 34.9   | 34.9   | 0.0 | 100.0 | 40.2      |

# Pemrograman Fungsional

- Pemrograman fungsional (*functional programming*) adalah salah satu paradigma pemrograman yang menekankan pada penggunaan **fungsi** sebagai **objek utama pemrograman**.
- Pemrograman fungsional dilakukan dengan menerapkan dan mengkomposisikan fungsi-fungsi.
- Setiap fungsi dianggap sebagai sebuah sub-komputasi yang memetakan sebuah input ke output tertentu.



# First-Class Object

Pada pemrograman fungsional, fungsi diperlakukan sebagai **first-class objects**.

- Fungsi dapat disimpan dan di-assign pada sebuah identifier, seperti halnya variabel.

```
val add = (x: Int, y: Int) => x + y
println(add(3, 4)) // Output: 7
```

- Fungsi dapat diberikan sebagai argumen dari fungsi lain.

```
def hitung(x: Int, y: Int, operation: (Int, Int) => Int): Int = {
    operation(x, y)
}

val result = hitung(5, 3, add)
println(result) // Output: 8
```

# First-Class Object

- Fungsi dapat mengembalikan fungsi yang lain.

```
def createMultiplier(factor: Int): Int => Int = {  
    (x: Int) => x * factor  
}  
  
val multiplyByTwo = createMultiplier(2)  
println(multiplyByTwo(4)) // Output: 8
```

First-Class Object memungkinkan alur pemrograman yang dapat dibuat secara **composable**: sebuah komputasi dilakukan dengan cara menggabungkan (komposisi) dari beberapa fungsi.

# Fungsi Murni

Pada pemrograman fungsional, biasanya diharapkan fungsi bersifat **pure functions** (**fungsi murni**).

- Fungsi hanya berperan untuk menerima input dan menghitung output.
- Input/argumen yang sama pasti memberikan output/keluaran yang sama (matematis dan deterministik)
- Tidak terpengaruh pada kondisi/state yang ada di luar fungsi.
- Tidak menyebabkan perubahan kondisi/state yang ada di luar fungsi (dengan kata lain, tidak memiliki side-effects / efek samping, termasuk ke memory/IO).

```
def hitung(a: Int): Int = {  
    3 * a + 5  
}  
  
val result = hitung(4)  
println(result)           // Output: 17
```

Fungsi **add** adalah murni karena selalu menghasilkan output yang sama untuk input yang sama, dan tidak menyebabkan efek samping apapun.



# Fungsi Murni

Berikut ini bukan fungsi murni. Mengapa?

```
def hitung(a: Int): Int = {  
    faktor * a + 5  
}
```

```
var faktor = 3  
var result = hitung(4)  
println(result)
```

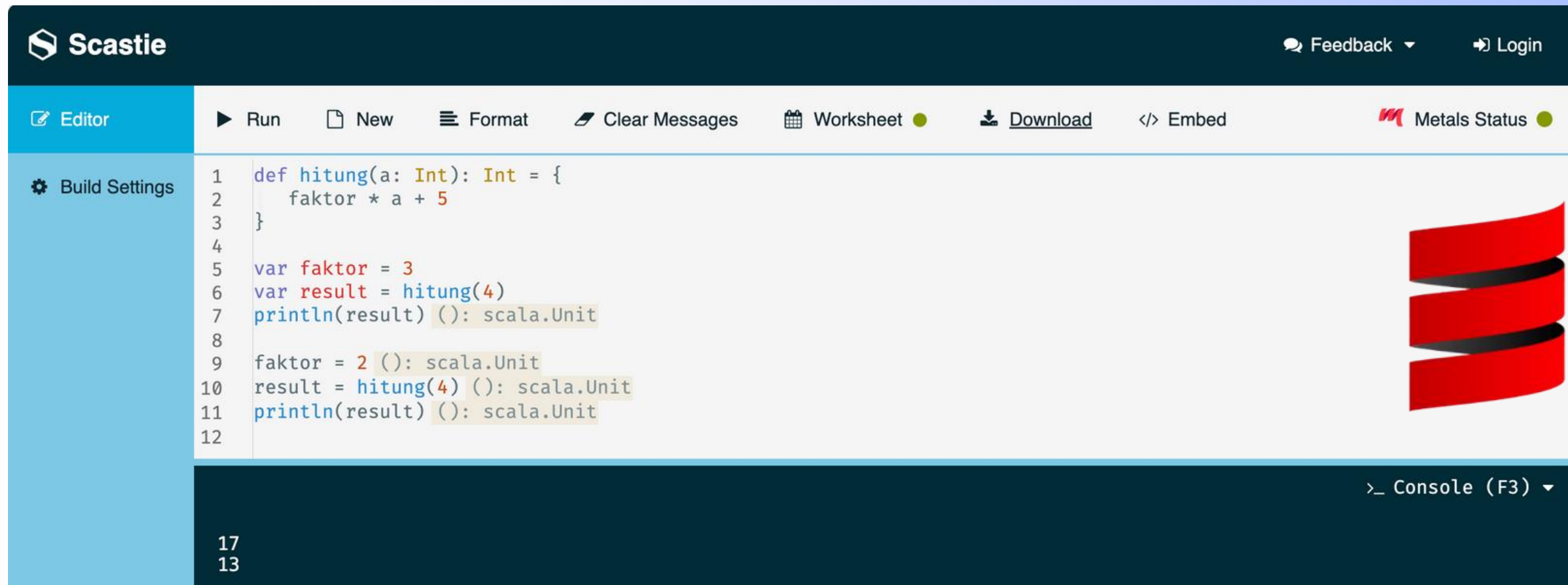
```
faktor = 2  
result = hitung(4)  
println(result)
```

- Perhatikan bahwa fungsi hitung dipengaruhi oleh sebuah variabel/ kondisi yang ada di luar fungsi (yaitu **faktor**).
- Nilai keluaran fungsi hitung tergantung tidak hanya pada argumennya (yaitu **a**), namun juga pada kondisi/state di luar fungsi.



# Pemrograman Scala

- Scala (**Sc**alable **La**nguage) adalah bahasa pemrograman high-level yang mendukung paradigma **object-oriented programming** dan **functional programming**.
- Dikembangkan pada tahun 2004 oleh Martin Odersky untuk mengatasi berbagai kelemahan yang ada pada Java
- Gunakan interpreter online pada <https://scastie.scala-lang.org/>



```
1 def hitung(a: Int): Int = {
2   faktor * a + 5
3 }
4
5 var faktor = 3
6 var result = hitung(4)
7 println(result) (): scala.Unit
8
9 faktor = 2 (): scala.Unit
10 result = hitung(4) (): scala.Unit
11 println(result) (): scala.Unit
12
```

17  
13



# Scala

# Variabel dalam Scala

- Tipe data: Byte, Short, Int, Long, Float, Double, Char, String, Boolean, **Unit**, Null.
- Scala mengenal dua jenis variabel:
  - Mutable → nilai dapat diubah-ubah | Deklarasi dengan **var**
  - Immutable → nilai tidak dapat diubah (konstan) | Deklarasi dengan **val**
- Variabel dapat dideklarasikan dengan tipe data secara eksplisit atau implisit.

```
// Eksplisit
var myVar1 : String = "Foo"

// Implisit
var myVar2 = 10           // Int
val myVal = "Hello Scala." // String
```

# Fungsi dalam Scala

- Sintaks fungsi dalam Scala:

```
def function_name ([parameter_list]) : [return_type] =  
    // function body
```

- Nilai kembalian fungsi adalah ekspresi terakhir yang ada pada fungsi tersebut.

```
def kuadrat(x: Int) : Int = x*x  
  
def sumAbs(a: Int, b: Int) : Int =  
    val absA = if a>0 then a else -a  
    val absB = if b>0 then b else -b  
    absA + absB  
  
println(kuadrat(4))  
println(sumAbs(5, -4))
```

# Input Output dalam Scala

- Mengeluarkan output dapat dilakukan dengan `println()`

```
var x:Int = 2
var y:Int = 5
println("x = "+x+", y = "+y+", x*y = "+x*y)
```



x = 2, y = 5, x\*y = 10

- Membaca nilai dari standard input menggunakan `scala.io.StdIn.readX()`

```
val a = scala.io.StdIn.readInt()
println("Nilai a = " + a)
```

Hal yang sama menggunakan fungsi-fungsi `readX()`

```
def readBoolean(): Boolean
def readByte(): Byte
def readChar(): Char
def readDouble(): Double
def readFloat(): Float
def readInt(): Int
def readLong(): Long
def readShort(): Short
```

```
def readLine(text: String, args: Any*): String
def readLine(): String
def readf(format: String): List[Any]
def readf1(format: String): Any
def readf2(format: String): (Any, Any)
def readf3(format: String): (Any, Any, Any)
```



# Struktur **IF-ELSE** dalam Scala

- Struktur kontrol **if-else** mirip dengan pada bahasa pemrograman imperatif lain

```
var x:Int = 70
if (x>=60) println("Lulus")
else println("Tidak Lulus")
```



Lulus

- Sebagai bagian dari fitur functional programming, Scala mendukung **expression-oriented programming** (EOP). Semua ekspresi (termasuk struktur if-else) pada Scala dapat mengembalikan nilai.
- Perhatikan pernyataan dalam fungsi nilai absolut sebelumnya di bawah ini:

```
def sumAbs(a: Int, b: Int) : Int =
{
    val absA = if a>0 then a else -a
    val absB = if b>0 then b else -b
    absA + absB
}
```

# Struktur **MATCH** dalam Scala

Struktur **match** digunakan untuk memilih dari banyak kasus (mirip seperti **switch case** pada C/C++)

```
import scala.util.Random

val x: Int = Random.nextInt(4)

val y = {
  x match {
    case 0 => "zero"
    case 1 => "one"
    case 2 => "two"
    case _ => "other"
  }
}

println(y)
```

# Struktur **WHILE** dalam Scala

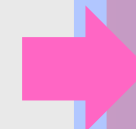
- Struktur kontrol perulangan dengan while pada Scala mirip dengan C/C++

```
while (boolean_expression)
{
    statement(s)
}
```

- Contoh

```
var i=0
while (i < 10)
{
    var kuadrat = i*i
    println(kuadrat)

    i+=1
}
```



```
0
1
4
9
16
25
36
49
64
81
```

# Struktur **FOR** dalam Scala

- Sintaks

```
for (a <- range) {  
    // set of statements  
}
```

- Contoh

```
var a = 0  
for (a <- 1 to 4) {  
    println( "a: " + a )  
}
```



```
a: 1  
a: 2  
a: 3  
a: 4
```

- Menggunakan generator : ekspresi yang menghasilkan banyak nilai dan dievaluasi secara lazy, artinya nilai yang diinginkan hanya akan dihasilkan ketika dibutuhkan

```
val gen = 1 to 4  
for (a <- gen) {  
    println( "a: " + a )  
}
```



```
a: 1  
a: 2  
a: 3  
a: 4
```



# Object dalam Scala

- Object dalam Scala merupakan **singleton**, artinya hanya ada satu instance.

```
object HelloWorld {    // Define an object
  def sayHello(): Unit = {    // Method
    println("Hello, World!")
  }
}
HelloWorld.sayHello() // Use the object
```

Hello, World!

- Object dalam Scala dapat juga berisi variables, methods, dan objects lainnya.

```
object Lingkaran {
  val PI: Double = 3.14
  def luas(radius: Double): Double = {
    PI * radius * radius
  }
}
```

```
val radius = 5.0
val area = Lingkaran.luas(radius)
println("Luas Lingkaran = " + area)
```

Luas Lingkaran = 78.5

# Object dalam Scala

- Fungsi main dalam object merupakan titik masuk (entry point) dari aplikasi.

```
object HelloWorld
{
  def main(args: Array[String]): Unit =
  {
    println("Hello, World!")
  }
}
```



Hello, World!

# Diskusi Kelas

---

Create a simple calculator that can perform basic arithmetic operations (addition, subtraction, multiplication, and division).

Instructions:

- Define a Scala object named **SimpleCalculator**.
- Create a method for each arithmetic operation:
  - > **add**(a: Double, b: Double): Double
  - > **subtract**(a: Double, b: Double): Double
  - > **multiply**(a: Double, b: Double): Double
  - > **divide**(a: Double, b: Double): Double
- Implement the methods to perform the respective operations.
- Create a main method to test your calculator. The main method should:
  - > Prompt the user to enter two numbers.
  - > Prompt the user to choose an operation (add, subtract, multiply, divide).
  - > Call the appropriate method based on the user's choice and display the result.